

ANEXA 5

Platforma Arduino. Introducere în implementarea interactivă a proiectelor electronice

Arduino este o platformă bazată pe o placă hardware ce conține un *microcontroller* (un circuit integrat îndeplinind funcțiile unui computer cu capacități reduse). De asemenea, Arduino conține și un *mediu de dezvoltare* integrat, prescurtat *IDE* (Integrated Development Environment), reprezentat de o componentă software accesibilă nemijlocit (*open source*), prin care utilizatorul poate comanda prototipul fizic, folosind programe intuitive, ușor de alcătuit, numite *sketch*-uri. În plus, versiunile mai noi de Matlab și Simulink permit conectarea cu partea hardware în mod automat, (metodă numită *rapid prototyping*), pentru care nu sunt necesare cunoștințe prealabile de programare.

Acest sistem poate servi la realizarea unor proiecte interactive simple, care să pună în evidență elementele electronice studiate, cu accent pe aplicații relevante din lumea reală. Utilizarea platformei Arduino poate contribui la impulsionarea învățării fundamentelor electronicii prin implementarea de prototipuri cu caracter practic. Printre avantajele aduse de această tehnică se numără: i) costul redus al plăcii Arduino; ii) accesibilitatea gratuită la *IDE* și la diferite scheme de prototipuri (*open source software* și *open source hardware*); iii) programarea microcontrollerului se face simplu, prin intermediul unui port *USB (Universal Serial Bus)* iv) existența unei comunități de utilizatori (forumuri pe web) a căror experiență poate fi fructificată; v) structura modulară ușurează înțelegerea, prin experiment, a unor sisteme mai complexe, actuale; vi) furnizează o privire de ansamblu a utilității practice a componentelor electronice învățate, contribuind la întărirea motivației în studiul electronicii. Toate acestea justifică pe deplin denumirea de ”prieten puternic” (Arduino) din limba italiană.

Placa Arduino

Conține un microcontroller (circuit integrat Atmel AVR ATmega328 cu 28 de pini), având capacități de calcul reduse (cam de 1000 de ori, față de un procesor utilizat într-un *PC*). Pe placă se mai găsesc și circuite de intrare/ieșire (*I/O* - input/output) pentru comunicarea cu exteriorul. În fig. A5.1 este prezentată structura unei plăci tipice, Arduino Uno.

Se observă existența a trei grupe de pini: *DIGITAL*, *ANALOG IN* și *POWER*. Din prima categorie fac parte 14 pini digitali, numerotați între 0 și 13, care pot fi setați prin

program ca intrări, sau respectiv ca ieșiri. În același timp, șase dintre acești pini, marcați cu *PWM* ~, (anume pinii 3,5,6,9,10 și 11) pot fi configurați ca ieșiri analogice. Cu ajutorul lor se pot converti valori cuprinse între 0 și 255, în tensiuni analogice.

În cea de-a doua categorie, se găsesc șase pini analogici de intrare (A0 – A5) care permit citirea semnalelor de la senzori, pentru a le converti apoi în numere binare între 0 și 1023. Grupul de pini din cea de-a treia categorie, a sursei de alimentare (ex. 3.3V, 5V, GND) pot servi la alimentarea unor componente externe de la placa Arduino. La rândul său, aceasta poate fi alimentată de la computer prin intermediul portului *USB* (tipic 5V, 500mA), sau de la o sursă externă (adaptor de c.a / baterie de 9V) folosind conectorul prevăzut în acest scop. De asemenea, comunicarea dintre *PC* și placa Arduino se realizează tot prin intermediul portului *USB*.

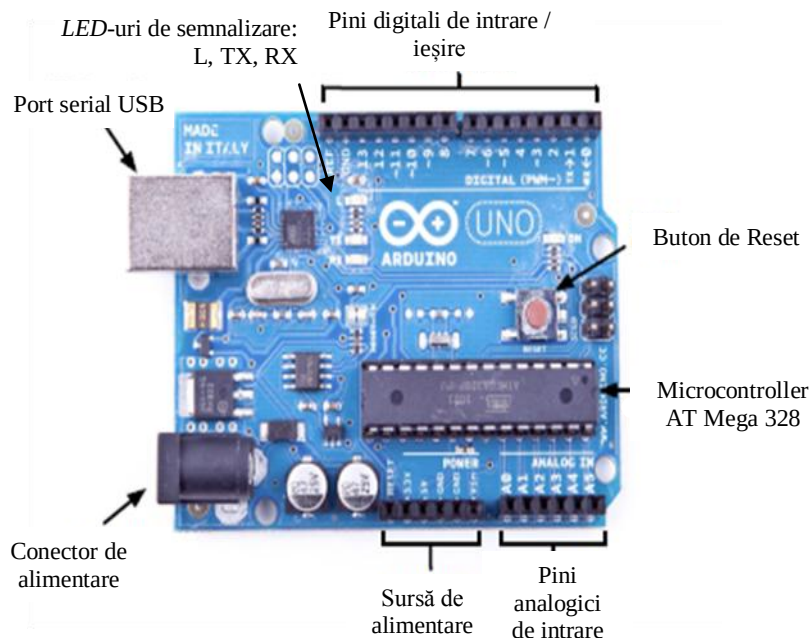


Fig. 5.1 Platforma Arduino Uno

Spre deosebire de calculatorul personal care își încarcă programele de pe hard-disk, un microcontroller trebuie să fie programat din exterior. Așadar, este necesară descărcarea, prin intermediul unui cablu, a setului de instrucțiuni ce trebuie efectuate. Odată ce programul a fost încărcat, el rămâne în microcontroller până când un nou set de instrucțiuni îi va lua locul, ștergându-le pe cele inițiale. Ori de câte ori placa Arduino este alimentată, ultimul program care a fost descărcat, se execută în mod automat. Butonul de *Reset* poate servi la inițializarea acestuia.

Acest sistem permite realizarea unei game variate de aplicații, care implică lucrul interactiv cu semnale, obiecte fizice din lumea reală. Termenul consacrat în literatură este cunoscut sub numele de *physical computing*. Pentru aceasta, anumite componente sunt atașate în exteriorul plăcii. Ele pot îndeplini funcția de *senzori* (convertesc o

mărimă fizică în semnal electric) sau pot avea rolul de a executa anumite comenzi (numite și *elemente de execuție* sau *actuatori*).

În acest sens, trebuie remarcat faptul că majoritatea proceselor fizice au la bază *semnale analogice* (ex. temperatura dintr-o încălț, presiunea atmosferică, semnalul acustic, etc.). Ele sunt transformate mai întâi în semnale electrice cu ajutorul traductoarelor sau *senzorilor*. Pe de altă parte, computerele, microcontrollerele, lucrează cu *semnale digitale*. Fig. A5.2 prezintă un exemplu de astfel de semnale. Deosebirea dintre ele constă în faptul că primele pot lua o infinitate de valori, pe când cele din urmă pot avea doar două valori :”0- logic” și ”1-logic” (ex. 0 V și respectiv +5V). De aceea, în practică este necesară conversia între cele două categorii de date.

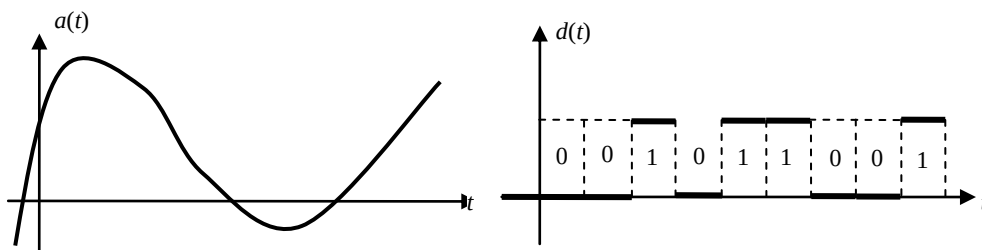


Fig. A5.2 Tipuri de semnale: analogice $a(t)$ și digitale $d(t)$

Transformarea semnalelor analogice în semnale digitale se realizează prin două operații: discretizarea pe axa timpului (*eșantionare*) și discretizarea pe axa amplitudinilor (*cuantizare*). Eșantionarea presupune memorarea valorilor semnalului la momente de timp egale (fig. A5.3).

Pasul cu care se face discretizarea pe axa timpului se numește *periodă de eșantionare* (notată de obicei cu T_s – “sampling time”). Semnalele eșantionate se mai numesc și *discrete*. Cuantizarea presupune împărțirea fiecărui eșantion în intervale mici numite cuante. Numărul acestora transformat în binar, este memorat și transmis sistemului de calcul.

Platforma Arduino are avantajul că poate efectua nenumărate conversii între cele două tipuri de semnale automat, fără a implica semnificativ cunoștințe deosebite ale utilizatorului în programarea microcontrollerelor. Se scurtează astfel timpul de proiectare, favorizându-se în schimb aplicarea

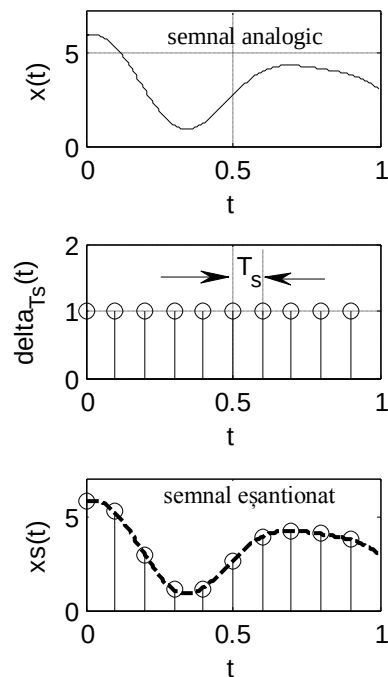


Fig. A5.3 Principiul eșantionării

a cât mai multor idei inovative în construirea prototipurilor.

Astfel, mărimea fizică măsurată este conectată prin intermediul unui senzor, la un pin analogic de intrare. Tensiunea analogică rezultată, este transformată în *semnal digital* cu ajutorul unui convertor analog/ digital (A/D), existent pe fiecare intrare. El poate citi semnale analogice cu tensiuni cuprinse între 0 și 5 V. Rezoluția convertorului A/D este de 10 biți. Aceasta înseamnă că amplitudinea fiecărui eșantion al tensiunii de intrare este discretizată (cuantizată) pe $2^{10} = 1024$ nivele. Așadar, fiecare eșantion este reprezentat intern printr-un număr cuprins între 0 și 1023, unde 0- corespunde nivelului masei 0V, respectiv 1023 corespunde nivelului maxim, de obicei 5V. Prin urmare, cea mai mică variație de tensiune care poate fi măsurată este $5V/1024 = 4.8mV$.

Pentru a produce un *semnal analogic* la ieșire, Arduino folosește tehnica PWM, o pseudo-metodă de conversie digital/analogică. Principiul acesteia este ilustrat în fig. A5.4. Denumirea de PWM (*Pulse Width Modulation* – Modulația Impulsurilor în Durată) provine de la faptul că la semnalul generat - în realitate digital – durata impulsului (intervalul de timp în care acesta este ”1” logic) poate fi modificată, concomitent cu păstrarea constantă a perioadei semnalului. Raportul procentual dintre durata impulsului T_p și perioada T a semnalului PWM, se numește *factor de umplere*, $u[\%]$:

$$u[\%] = \frac{T_p}{T} \cdot 100 \quad (A5.1)$$

Astfel, prin modificarea factorului de umplere, se poate varia *valoarea medie* X_M a semnalului generat (fig. A5.4). Deoarece frecvența semnalului PWM este relativ mare (circa 500 Hz), practic elementele de execuție conectate la ieșire (ex. dispozitive de afișare -LED-uri, dispozitive audio, elemente de acționare mecanică- motoare, etc.) îl vor interpreta ca pe un semnal analogic de valoare egală cu *media* semnalului digital PWM aplicat. În fapt, prin modificarea duratei impulsurilor semnalului, se modifică puterea livrată sarcinii la ieșire.

Aplicând definiția valorii medii (v. Anexa 2, relația A2.3.4), în acest caz se obține:

$$X_M = \frac{1}{T} \cdot \int_0^T x(t) dt = \frac{1}{T} \cdot \int_0^{T_p} U dt = \frac{T_p}{T} \cdot U \quad (A5.2)$$

unde U este tensiunea corespunzătoare valorii ”1”- logic (nivel HIGH, de obicei 5V). De asemenea, perioada T semnalului PWM este aproximativ 1/500 Hz, adică 2 ms. De exemplu, pentru un factor de umplere $u=14\%$, durata impulsului este $T_p=0.28ms$, iar valoarea medie a va fi $u \cdot U/100 \approx 0.7 V$ (fig. A5.4)

La sistemul Arduino Uno, semnalul PWM poate fi produs la pinii 3,5,6,9,10 și 11. Pentru a genera o tensiune analogică, se specifică o valoare din intervalul 0-255. Această valoare corespunde factorului de umplere a impulsului PWM (0 pentru 0% - reprezentând 0V mergând până la 255 pentru 100%, reprezentând 5V). În acest scop se folosește instrucțiunea

$$\text{analogWrite (pin_PWM, valoare);} \quad (A5.3)$$

unde `pin_PWM` reprezintă numărul pinului *PWM* configurat ca ieșire analogică, iar variabila `valoare` este un număr cuprins între 0 și 255, așa cum s-a arătat anterior. De exemplu, pentru a obține cei 0.7 V de mai sus, trebuie specificat cel mai apropiat număr întreg din intervalul 0-255, ales proporțional - aici $0.7 \cdot 255/5=35.7$ prin urmare se poate adopta valoarea 36.

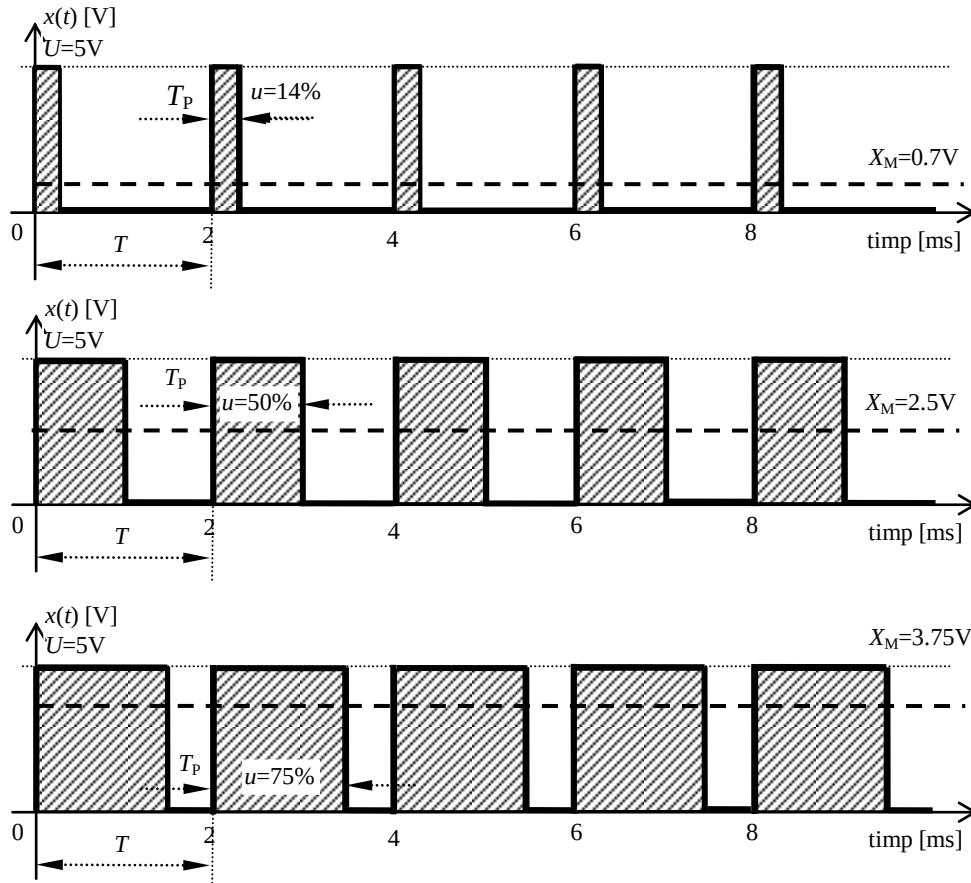


Fig. A5.4 Principiul PWM

Componenta software - Arduino IDE

Programele care urmează a fi executate pe platforma Arduino (numite *sketch-uri*) sunt create pe un computer, folosind mediul său de dezvoltare integrat, prescurtat *IDE* (*Integrated Development Environment*) - o colecție de pachete software accesibile gratuit (*open source*). Acesta este ușor de instalat, fiind compatibil cu majoritatea sistemelor de operare (ex. pașii pentru Windows sunt prezentați în Anexa 1, Tabelul A1.19). Prin intermediul său, utilizatorul poate edita programele care apoi sunt

convertite în cod mașină (*compile*) și încărcate (*upload*) pe platforma Arduino pentru a fi executate (fig. A.5.5). Evident, pe tot parcursul implementării proiectului, programul poate fi modificat și ajustat, astfel încât acesta să îndeplinească cerințele date.

În fig. A5.6 este prezentată interfața tipică de editare folosită în acest scop. După conceperea *sketch*-ului și salvarea acestuia, se face click pe butonul de compilare. Starea curentă a desfășurării procesului

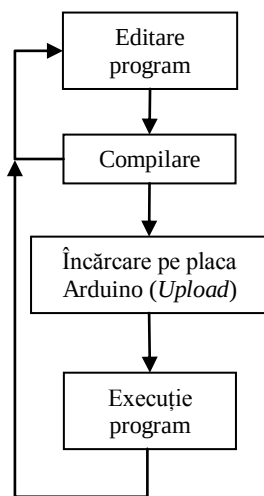


Fig. A5.5 Etape de dezvoltare a unui proiect interactiv folosind Arduino

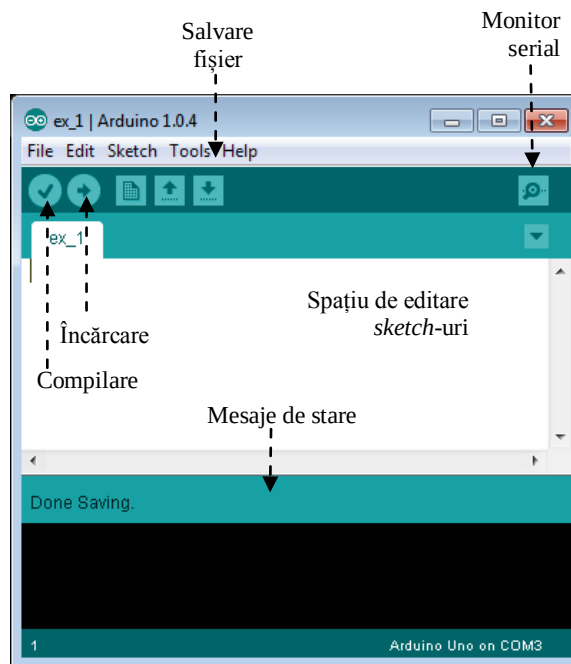


Fig. A5.6 Fereastra de editare Arduino IDE

de implementare este afișată în partea de jos a ferestrei IDE. În cazul în care nu există erori, se trece la încărcarea codului obiect pe platforma Arduino. LED-urile TX / RX (fig 5.1), vor semnaliza momentele de transmisie a programului de către PC, respectiv de recepție a acestuia la placă. Concomitent, în zona cu fundal negru, se afișează și un mesaj care indică spațiul de memorie ocupat de program.

De remarcă faptul că Arduino IDE dispune de numeroase modele de *sketch*-uri, foarte utile pentru inițierea în utilizarea acestei platforme și implementarea de proiecte interactive. Ele se găsesc în meniul *File* → *Examples*. Cel mai simplu exemplu, prin intermediul căruia se poate face și verificarea funcționării plăcii, este denumit *Blink* și se află în meniul *01.Basics*. El comandă aprinderea și stingerea LED-ului L conectat intern la pinul 13. Setul de instrucțiuni care realizează acest lucru, este prezentat în fig. A5.7. Comenzile Arduino au la bază limbajul de programare C/ C++, cu mențiunea că acestea au fost simplificate într-un format intuitiv, prietenos și ușor de utilizat.

Structura unui program și o listă cu instrucțiuni tipice sunt prezentate în Anexa 1, Tabelele A1.20-A1.30.

```

/*
  Blink
  Turns on an LED on for one second, then off for one second, repeatedly.

  This example code is in the public domain.
*/

// Pin 13 has an LED connected on most Arduino boards.
// give it a name:
int led = 13;

// the setup routine runs once when you press reset:
void setup() {
  // initialize the digital pin as an output.
  pinMode(led, OUTPUT);
}

// the loop routine runs over and over again forever:
void loop() {
  digitalWrite(led, HIGH); // turn the LED on (HIGH is the voltage level)
  delay(1000);             // wait for a second
  digitalWrite(led, LOW);  // turn the LED off by making the voltage LOW
  delay(1000);             // wait for a second
}

```

Fig. A5.7 Blink: model simplu de program pentru comanda unui LED

În exemplul din fig. A5.7, textele demarcate de caracterele `/*`, `*/`, și `//` reprezintă explicații referitoare la ce se realizează prin program, respectiv rolul unei comenzi, sau a unui bloc de instrucțiuni, etc (v. Anexa 1, Tabelul A 1.22). Deși aceste comentarii nu sunt interpretate de compilator, se recomandă călduros scrierea lor, deoarece ele permit diferiților utilizatori să aibă o mai bună înțelegere a programului și eventual să-l ajusteze conform aplicației concrete.

Prima instrucțiune din exemplul de mai sus (`int led = 13;`), introduce variabila `led`, căreia îi atribuie valoarea 13, corespunzătoare pinului digital la care este conectat *elementul de execuție* (în cazul de față un dispozitiv de afișare - o diodă electroluminiscentă *LED (Light Emitting Diode)*, notată cu *L* aflată pe placa Arduino, așa cum se observă în fig. A5.1). (Se observă că spre deosebire de *LED*-urile obișnuite, cu terminale, folosite în implementarea de prototipuri, *LED*-ul încorporat pe placa Arduino este o variantă în miniatură, fiind realizat în tehnologia *SMT - Surface Mounted Technology*)

Trebuie menționat faptul *LED*-urile sunt dispozitive electronice foarte des utilizate în multe aplicații practice, în scopul semnalizării, afișării, transmiterii informației, etc. Pe scurt, atunci când ele sunt alimentate cu o tensiune de o anumită valoare și o

anumită polaritate, emit o radiație luminoasă. Frecvența radiației („culoarea” luminii *LED*-ului) depinde de materialul din care acestea sunt construite (ex. GaAs pentru cele roșii). *LED*-urile au la bază același principiu de funcționare ca și diodele semiconductoare. Cele două terminale denumite *anod* și *catod*, se conectează prin intermediul unei rezistențe la bornele tensiunii de alimentare (fig. A5.8). Dacă acesta este *polarizat direct* (adică plusul bateriei este legat la anod și minusul la catod), și dacă tensiunea aplicată este mai mare decât *tensiunea de prag* a *LED*-ului - specificată în catalog - el se va deschide, emițând o radiație luminoasă. (Ex. Tipic, pentru un *LED* roșu, trebuie să se asigure la bornele sale o tensiune de cel puțin 1.8V). Se poate arăta că în această situație curentul prin circuit va crește exponențial, iar lipsa unei rezistențe înseriate cu el va produce distrugerea acestuia. După cum s-a menționat anterior, platforma Arduino Uno conține *LED*-ul *L*, înseriat intern cu o *rezistență de limitare* (tipic având valoarea 220Ω) la pinul 13.

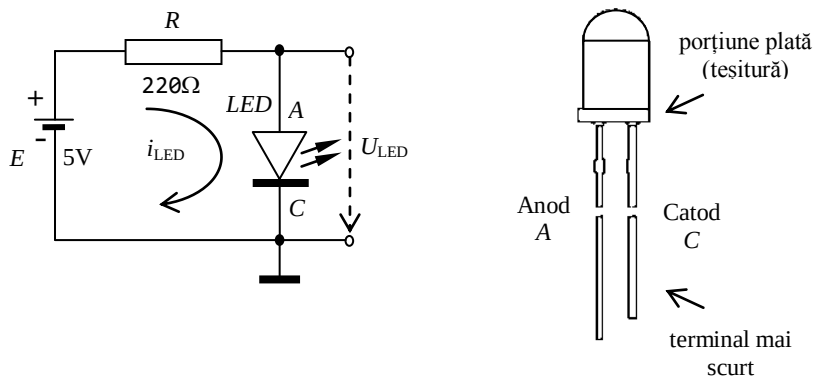


Fig.A5.8 a) Circuit de polarizare a unui *LED*; b) Aspectul fizic al unui *LED* (tehnologia cu terminale – *Through Hole Technology* - THT)

Revenind la prima instrucțiune din exemplul ilustrat în fig. A5.7, prefixul `int` arată că variabila `led` este declarată ca mărime întreagă (v. Anexa 1 Tabelul A1.23). Programul continuă cu funcțiile specifice **setup** și **loop** (v. Anexa 1, Tabelul A1.21), obligatorii într-un sketch Arduino. Așa cum se descrie și în comentariu, rutina **setup** rulează o singură dată, la acționarea butonului de *reset*, și realizează aici setarea pinului 13 ca ieșire (v. Anexa 1 Tabelul A1.30). Instrucțiunile din corpul funcției **loop** comandă mai întâi generarea la pinul 13 a unui nivel de tensiune HIGH (de obicei 5V), care va produce aprinderea *LED*-ului și menținerea lui în această stare timp de 1000 milisecunde (1 secundă). Apoi se va comanda generarea la același pin a unui nivel de tensiune LOW (0 V), care va stinge *LED*-ul timp de o secundă. Această rutină se va derula în mod continuu, atâta timp cât placa Arduino va fi alimentată.

Pornind de la acest exemplu tipic, utilizatorul poate introduce propriile ajustări, în funcție de cerințele concrete întâlnite în practică. Un prim lucru ce se poate face este modificarea parametrului specificat în instrucțiunea `delay`, care va comanda timpul pentru care *LED*-ul va rămâne aprins sau stins.

Același program se poate folosi și în cazul în care se dorește comanda unui *LED* montat în exteriorul plăcii, eventual la un alt pin. În acest sens, se realizează mai întâi circuitul electric conform diagramei din fig. A5.9, apoi se încarcă și execută programul de mai sus, unde eventual la prima instrucțiune s-a modificat în prealabil numărul pinului, astfel încât acesta să corespundă cu cel la care este conectat dispozitivul de afișare. Un prototip fizic pentru acest model este prezentat în fig. A5.10.

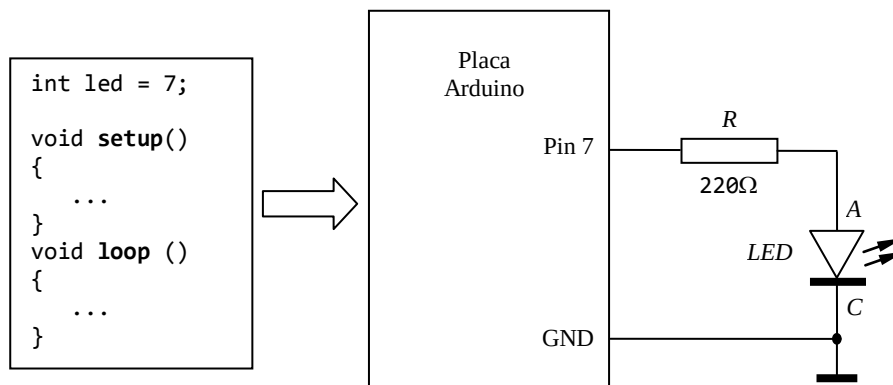


Fig.A5.9 Ajustarea modelului *Blink* pentru un *LED* extern, conectat la un alt pin Arduino

De menționat faptul că acest exemplu poate fi implementat ușor și cu ajutorul mediului de programare Matlab/ Simulink. Astfel, se poate înlocui *sketch*-ul Arduino cu un fișier ce conține blocuri Simulink, ca cel din fig. A5.11a, unde timpii pentru care *LED*-ul este rămane aprins, respectiv stins, sunt configurați în blocul *Pulse Generator*. Pentru încărcarea programului Simulink pe platforma Arduino se folosește opțiunea *Deploy to HardWare*, conținută în pachetul Simulink, similară cu operațiunea de *Upload* oferită de Arduino *IDE*. De asemenea, în Fig. A5.11b și c se prezintă și configurațiile parametrilor modelului Simulink, setările pentru modulul *Pulse Generator*, și două exemple de forme de undă afișate prin intermediul blocului *Scope* pentru comanda *LED*-ului.

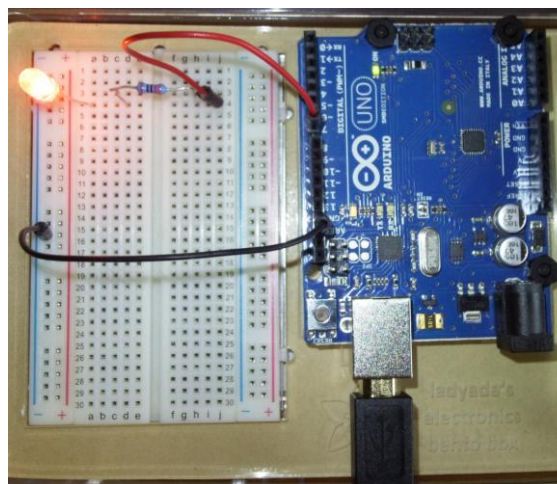


Fig. A5.10 Circuit fizic pentru exemplul *Blink* cu *LED* conectat în exteriorul plăcii

În concluzie, indiferent de metoda folosită pentru implementarea prototipului prezentat mai sus, se obține același rezultat: acționarea și controlul dispozitivului fizic atașat extern. Utilizatorul poate opta pentru o variantă de implementare sau alta: prin programare, folosind *Arduino IDE*, printr-o metodă ”vizuală”, folosind scheme bloc, sau ambele metode.

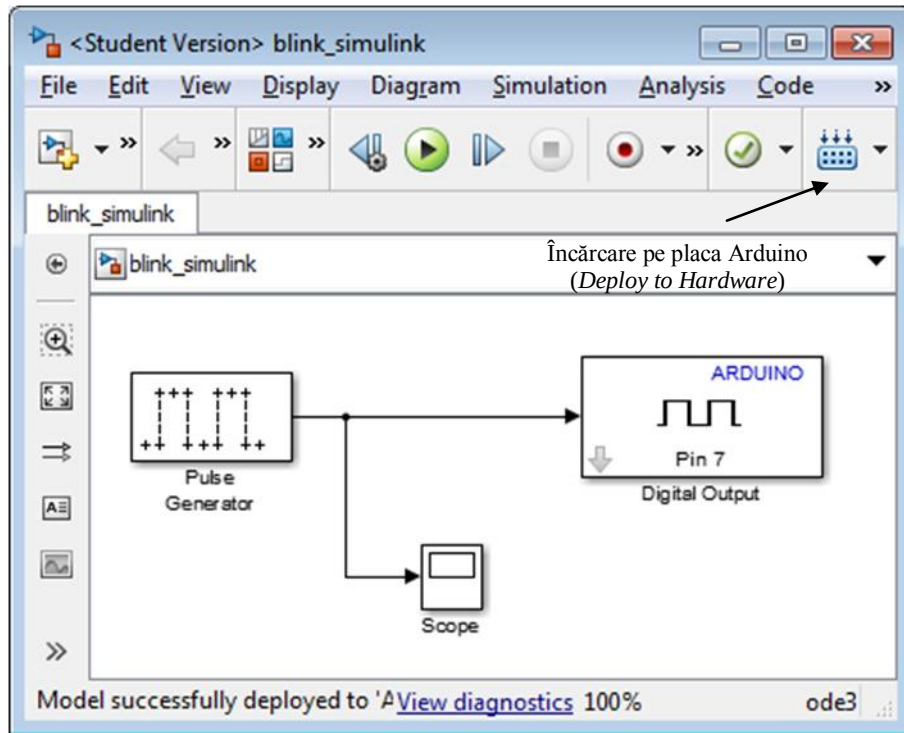


Fig. A 5.11 a) Schema bloc Simulink pentru modelul *Blink*

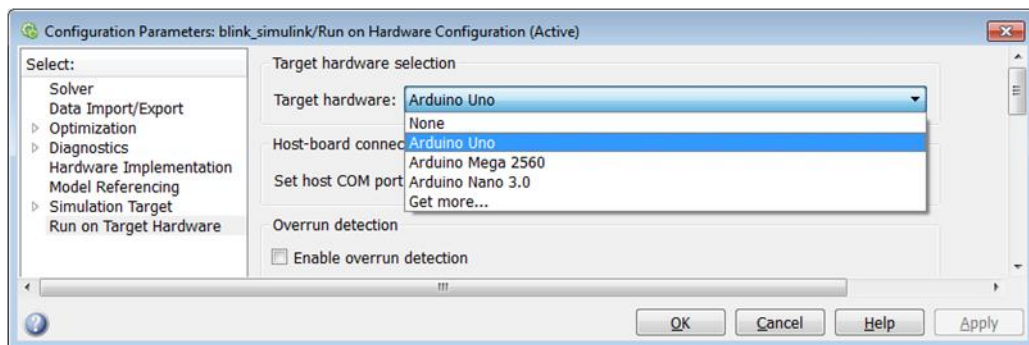


Fig. A5.11 b) Configurarea parametrilor pentru execuția modelului *Blink* pe placa Arduino

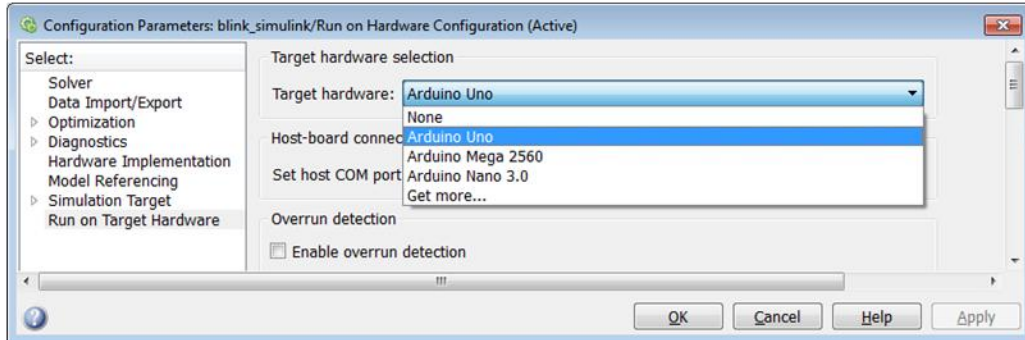


Fig. A5.11 b)- continuare- Configurarea parametrilor pentru execuția modelului *Blink* pe placa Arduino

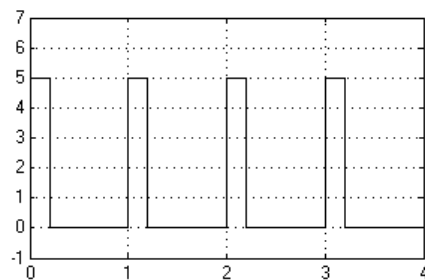
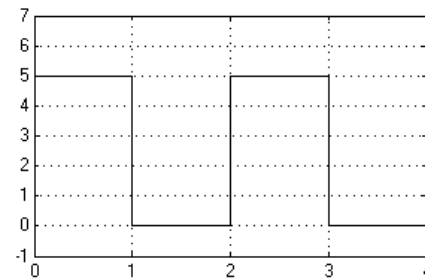
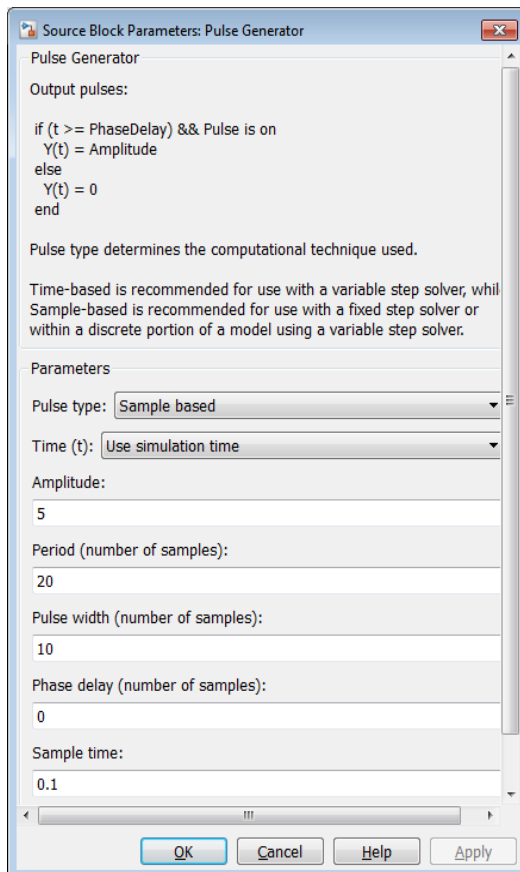


Fig. A5.11 c) Modul de configurare a blocului generator de impulsuri prin care se apacă timp de aprindere/ stingere a *LED*-ului (stânga); Exemple de forme de undă pentru comanda *LED*-ului (dreapta)

Folosind același circuit fizic ca cel fig. A5.9 - A5.10, se poate realiza un model experimental prin care să se comande strălucirea *LED*-ului. Pentru aceasta, se modifică doar pinul la care este conectat *LED*-ul, astfel încât acesta să fie de tip *PWM*, și programul aferent, care va conține comanda `analogWrite` (v. Anexa 1, Tabelul A1.30). Fig. A5.12 indică un exemplu de *sketch* prin care se crește apoi descrește treptat strălucirea *LED*-ului. Tot aici se demonstrează și utilizarea instrucțiunii de control `for` (v. Anexa 1, Tabelul A1.28).

```
/* Program pentru demonstrarea utilizarii iesirii analogice PWM;
Elementul de executie este un LED conectat la pinul 3,
a carui stralucire este variata prin program */

int led=3;// LED-ul este conectat la pinul 3, de tip PWM

void setup(){ // Nu exista setari suplimentare

void loop()
{
  for (int i=0; i<=255; i++)// variabila i crescatoare, determina
  {
    // factorul de umplere PWM si ca urmare
    analogWrite(led,i); // stralucirea LED-ului creste in intensitate
    delay(30);          // Mentinere 30 ms
  }
  for (int i=255; i>=0; i--) // variabila i descrescatoare, face ca
  {
    // factorul de umplere PWM sa scada, si ca
    urmare
    analogWrite(led,i); // stralucirea LED-ului scade treptat in
    intensitate
    delay(30); // Mentinere 30 ms
  }
}
```

Fig. A5.12 Program pentru comanda strălucirii *LED*-ului

Echivalent, se poate implementa aceeași funcție, folosind o schemă Simulink ca în fig. A5.13a), unde intensitatea strălucirii *LED*-ului este dată de blocul generator pus la intrare, în acest caz un modul *Repeating Sequence*, configurat ca în fig. A5.13b. Astfel durata creșterii, respectiv descreșterii luminozității elementului de afișare trebuie configurată corespunzător pentru a obține același efect. Pentru exemplul de mai sus, ea trebuie să fie egală cu timpul de menținere la același nivel de strălucire (notat cu `delay` în *sketch*-ul Arduino), transformat în secunde, înmulțit cu numărul total de valori analogice comandate la ieșire (aici 255). De asemenea, pasul de eșantionare (*Fixed- Step Size*) se poate alege egal cu perioada semnalului *PWM*, adică aproximativ $1/500 \text{ Hz} \approx 2 \text{ ms}$.

În mod similar, blocul *Repeating Sequence* din schema de mai sus, se poate înlocui și cu alte module din biblioteca Simulink a surselor (v. Anexa 1 Tabelul A1.18a), ca *Sine Wave*, *Random Number*, *Signal Generator*, etc. În acest fel se pot obține o multitudine de efecte vizuale.

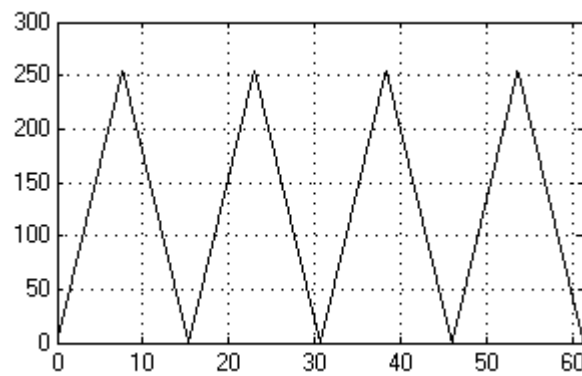
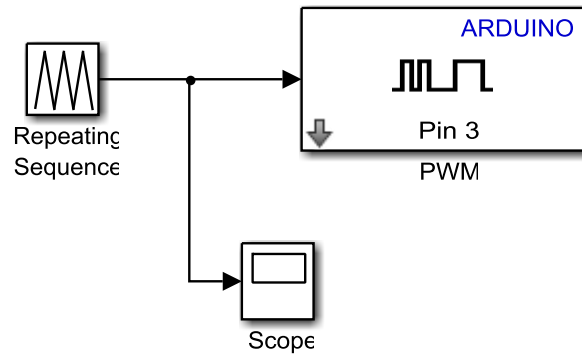


Fig. A5.13 a) Schema Simulink echivalentă cu sketch-ul Arduino din fig. A5.12 (sus);
Forma de undă a secvenței periodice de comandă (jos)

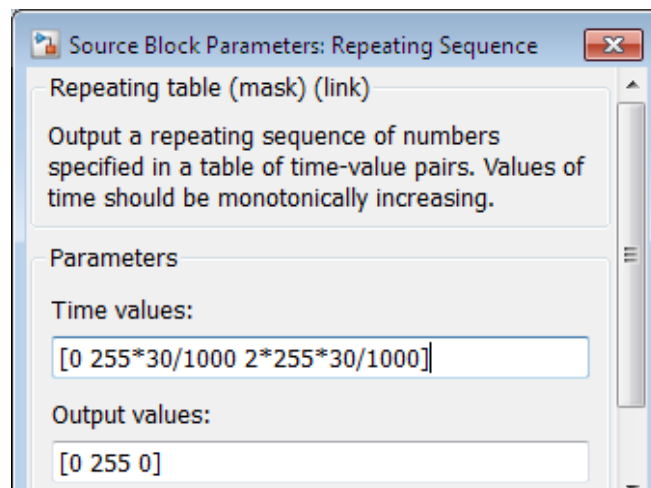


Fig. A5.13 b) Configurarea blocului *Repeating Sequence* pentru digrama din fig. A5.13 a)

De asemenea, comandând în aceeași manieră trei *LED*-uri diferite, anume roșu, verde și albastru - sau echivalent un *LED RGB*, care le înglobează pe toate trei (*R*-roșu, *G*-verde, *B*-albastru) - se pot face multiple combinații de culori.

În multe aplicații practice, acționarea elementelor de execuție este condiționată de valorile semnalelor, culese de senzori, la intrările sistemului.

Un prim exemplu de acest tip, implementat ușor pe platforma Arduino, este comanda unei ieșiri digitale, condiționată de starea unei intrări digitale (fig. A5.14a). Aici senzorul este un simplu comutator *K*, ce poate îmbrăca diferite forme: normal deschis (*pushbutton*), normal închis – (ex. cu bilă - *tilt switch*), magnetic, etc. De pildă comutatorul cu bilă, ar putea fi montat pe un suport într-o instalație, deschizându-se și întrerupând circuitul, în cazul în care suportul pe care acesta se află s-ar înclina peste un anumit unghi, declanșând la rândul său o alarmă.

Cu ajutorul unui program simplu, ca cel din fig. 5.14b (*sketch* Arduino), sau ca cel ilustrat în fig. 5.14c (schemă Simulink), compilat și încărcat în microcontroller, se poate citi starea comutatorului, și în funcție de aceasta, se poate comanda pornirea/oprirea semnalizării *LED*-ului conectat la ieșire.

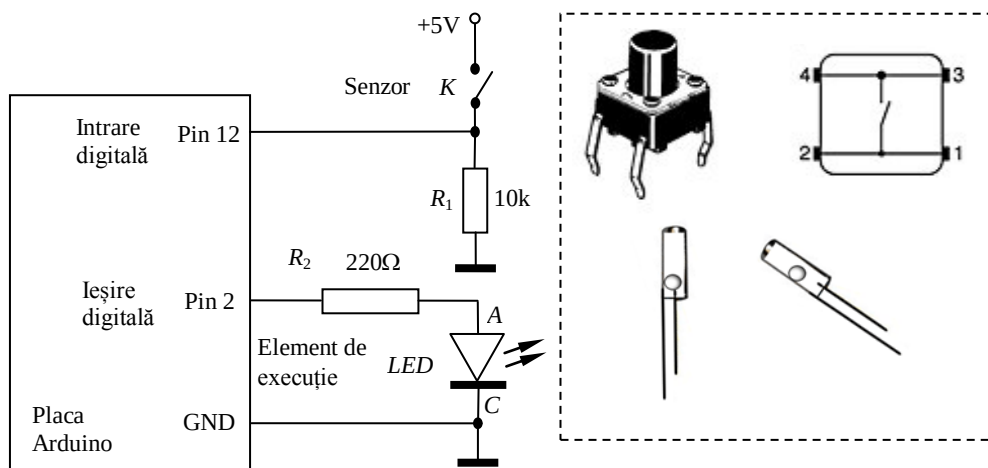


Fig.A5.14a) Circuit fizic pentru comanda unei ieșiri digitale, condiționată de o intrare digitală (stânga); exemple de senzori cu două stări: întrerupător normal deschis (*pushbutton* -dreapta sus), normal închis - întrerupător cu bilă (*tilt switch* - dreapta jos)

În *sketch*-ul din fig. A5.14b, se observă și modul de utilizare a instrucțiunii de control *if* (v. Anexa 1, Tabelul A1.28), în acest caz opțiunile *elseif* și *else* nefiind necesare. Prin intermediul acesteia se testează dacă nivelul de tensiune de la pinul 12, configurat aici ca intrare, este + 5V (*HIGH*), lucru echivalent cu sesizarea închiderii comutatorului *K*. În această situație, se comandă clipirea *LED*-ului la intervale de 1 secundă. În caz contrar, intrarea este pusă la masă (nivel logic *LOW*) prin intermediul rezistenței *R₁* (În lipsa acestei rezistențe, nivelul tensiunii de intrare ar fi nedeterminat, arbitrar între 0 și 5 V).

În varianta Simulink din fig. 5.14c, ieșirea blocului *Digital Input* este ”1”- logic dacă nivelul potențialului pinului de intrare specificat corespunde stării HIGH (ex. +5V), și ”0”- logic dacă acesta corespunde stării LOW (ex. 0V). Evident, la ieșirea blocului produs (*Product*) se va obține semnalul de comandă a LED-ului - dat de generatorul de impulsuri - doar dacă la intrare se va citi starea HIGH.

```
/* Program pentru demonstrarea comenzii unei iesiri digitale
conditionata de o intrare digitala */

int led = 2;    // LED-ul (elementul de executie) este legat la pinul 2
int pushButton = 12; // Intrerupatorul (senzorul) conectat la pinul 12

void setup()
{
    pinMode(led,OUTPUT); // Se configureaza pinul conectat la LED ca iesire
    pinMode(pushButton,INPUT); // si pinul conectat la intrerupator ca intrare
}

void loop()
{
    int state = digitalRead(pushButton); // Se citeste starea intrarii si se
                                         // memoreaza in variabila state
    if (state == HIGH) // Daca butonul este apasat
    {
        digitalWrite(led, HIGH); // Se comanda clipirea LED-ului
        delay(1000); // la intervale de 1000 milisecunde
        digitalWrite(led, LOW);
        delay(1000);
    }
}
```

Fig. A 5.14b) Exemplu de comandă a unei ieșiri digitale, condiționată de o intrare digitală

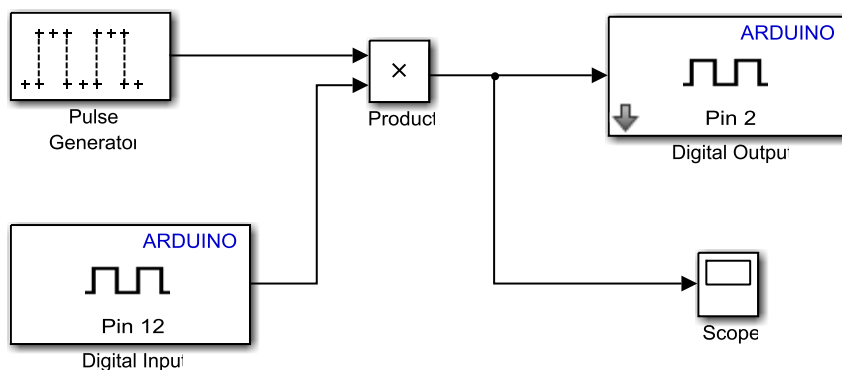


Fig. A5.14 c) Schema Simulink echivalentă pentru sketch-ul din fig. 5.14 b)

Folosind același circuit fizic (fig. A5.14a), se poate completa programul din exemplul precedent, astfel încât să se combine cele două tipuri de comenzi prezentate mai sus, pentru acționarea ieșirii. De pildă *LED*-ul poate fi configurat ca ieșire digitală –în cazul în care comutatorul *K* este închis, producând un efect de clipire, sau ca ieșire analogică în caz contrar, obținându-se varierea luminozității prin tehnica *PWM*. Codul Arduino și schema echivalentă Simulink sunt prezentate în fig. A5.15. Se observă utilizarea opțiunii *else* în construcția *if* (fig. A5.15a). Funcția acestei instrucțiuni de control poate fi realizată echivalent de blocul *Switch* din Simulink. Acesta lasă să treacă la ieșire fie semnalul produs de modulul *Pulse Generator*, fie semnalul generat de blocul *Repeating Sequence*, în funcție de valoarea de adevăr (*T*-True-adevărat / *F*-False - fals) a condiției specificată la intrarea *Arduino Digital Input*.

```
/* Program pentru actionarea unei iesiri digitale sau analogice
conditionata de o intrare digitala */

int led = 3; //LED-ul (elementul de executie) este legat la pinul 3 (PWM)
int pushButton = 12; // Intrerupatorul (senzorul) conectat la pinul 12

void setup()
{
  pinMode(led,OUTPUT); //Se configureaza pinul conectat la LED ca iesire
  pinMode(pushButton,INPUT); // si cel conectat la intrerupator ca intrare
}

void loop()
{
  int state = digitalRead(pushButton); // Se citeste starea intrarii
  if (state == HIGH) // Daca butonul este apasat
  {
    digitalWrite(led, HIGH); // Se comanda clipirea LED-ului
    delay(30); // la intervale de 30 milisecunde
    digitalWrite(led, LOW);
    delay(30);
  }
  else // Daca butonul nu este apasat, se comanda LED-ul prin tehnica PWM
  {
    for (int i = 0; i <= 255; i+=5) // Se creste luminozitatea cu pasul 5
    {
      analogWrite(led, i);
      delay(30); // Mentinere la nivelul curent timp de 30 ms
    }
    for (int i = 255; i >= 0; i-=5) //Apoi luminozitatea se scade
    {
      analogWrite(led,i); // in aceeași maniera
      delay(30);
    }
  }
}
```

Fig. A 5.15a Ieșire digitală sau analogică în funcție de starea intrării digitale

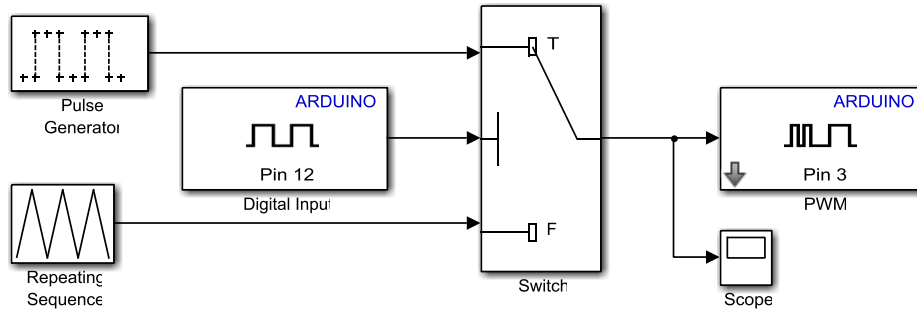


Fig. A5.15 b) Schema Simulink echivalentă pentru sketch-ul din fig. 5.15a)

O altă clasă de aplicații importante care pot fi implementate relativ ușor pe platforma Arduino, este comanda în timp real a unei ieșiri digitale, condiționată de valoarea semnalului *analogic* de la intrare. Un exemplu simplu din acest tip este ilustrat în fig. A5.15 a), unde la una dintre intrările analogice (ex. A0) se conectează cursorul unui potențiomtru. Valoarea tensiunii de intrare poate fi reglată în acest fel, între 0 și tensiunea de alimentare (ex. 5V). Prin intermediul convertorului analog digital (A/D) existent pe placă, pentru intrarea analogică aleasă, această tensiune este citită și reprezentată intern printr-o valoare cuprinsă între 0 și 1023. Programul implementat și descărcat pe platforma Arduino, va da comandă *LED*-ului conectat la la ieșirea digitală, să clipească în ritmul unei secvențe specificate de utilizator.

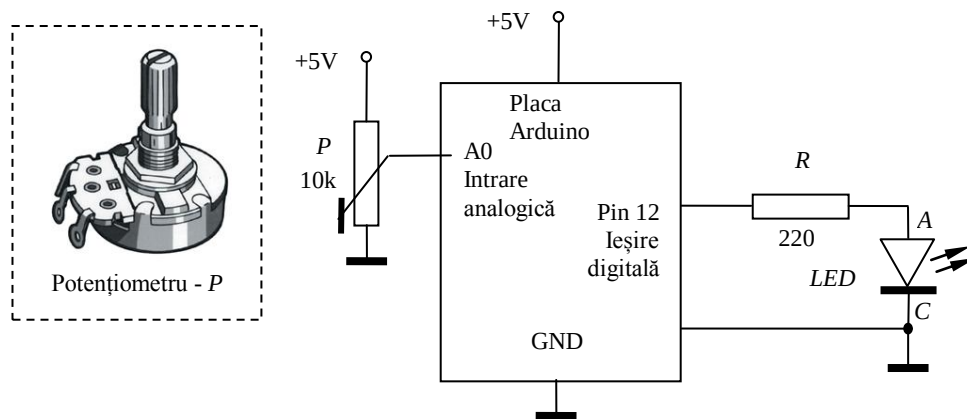


Fig.A5.16a) Cicuit fizic pentru comanda unei ieșiri digitale, condiționată de o intrare analogică (dreapta); Sensor analogic elementar (*potențiomtru* -stânga)

De exemplu, se poate varia frecvența de aprindere/stingere a acestuia în funcție de poziția potențiometrului. Programul Arduino care implementează acest lucru este prezentat în fig. A5.1 6.b, unde după declararea variabilelor și configurarea ieșirii, în secțiunea **loop()**, la fiecare parcurgere a buclei, se citește valoarea tensiunii prescrise

de la potențiometrul. Concomitent, se comandă menținerea alternativă la ieșire a valorilor logice "1" respectiv "0" un timp egal în milisecunde cu valoarea citită la intrare, având ca efect aprinderea/stingerea LED-ului cu o frecvență reglabilă.

```
/* Program pentru demonstrarea utilizarii intrarii analogice:
Comanda vitezei de clipire a unui LED conditionata de nivelul analogic
de la intrare*/

int led = 12; // Pin la care este conectat LED-ul
int pot = 0;  // Pinul analogic de intrare A0 la care este legat
              //potentiometrul

void setup()
{
  pinMode (led, OUTPUT);    // pinul 12 se configureaza ca iesire
}
void loop()
{
  int u = analogRead(pot); // Tensiunea u este citita si convertita
                          //intr-un numar intre [0,1023]
  digitalWrite (led, HIGH); // LED-ul va clipi
  delay(u);              // la intervale prescrise de potentiometru (ms)
  digitalWrite (led, LOW); // proportional cu tensiunea u citita la
                          //intrare
  delay(u);
}
```

Fig. A5.16 b Sketch Arduino pentru experimentul din fig. A5.16 a

Evident, această aplicație se poate implementa și cu ajutorul Matlab-ului. O variantă de schemă Simulink pentru exemplul de mai sus, este prezentată în fig. A5.16c. În general, atunci când se proiectează o schemă Simulink, este indicat ca mai întâi să se utilizeze blocuri/ funcții interne, așa cum se observă în fig. A5.16c. Acest lucru ajută la testarea principiului de funcționare a modelului. Pentru exemplul ales, în locul potențiometrului se utilizează un bloc cu funcție similară, "Slider Gain", care poate fi reglat în timpul simulării. De asemenea, pentru a vizualiza secvențele de la ieșire se poate utiliza un bloc "Scope". În fig. A5.16e, este prezentată o situație de simulare a modelului de mai sus. În acest caz, principiul ales se bazează pe implementarea unui numărător, care se resetează atunci când se atinge dublul nivelului prescris de potențiometrul. Semnalul rezultat (notat $y++$) va avea perioada $2 \cdot \text{prag}$. El este apoi centrat, obținându-se o secvență cu media nulă ($y++$ centrat). Cu ajutorul unui bloc selector, se comandă la ieșire +5V dacă secvența ia valori pozitive, respectiv 0V pentru valori negative. În acest fel, modelul descris cu ajutorul Simulink implementează aceeași funcție ca și programul Arduino descris în fig. A5.16b. Atunci când rezultatele de simulare satisfac cerințele de proiectare și nu apar erori, se poate

realiza înlocuirea blocurilor interne cu modulele specifice din toolboxul Arduino (fig. A5.16d).

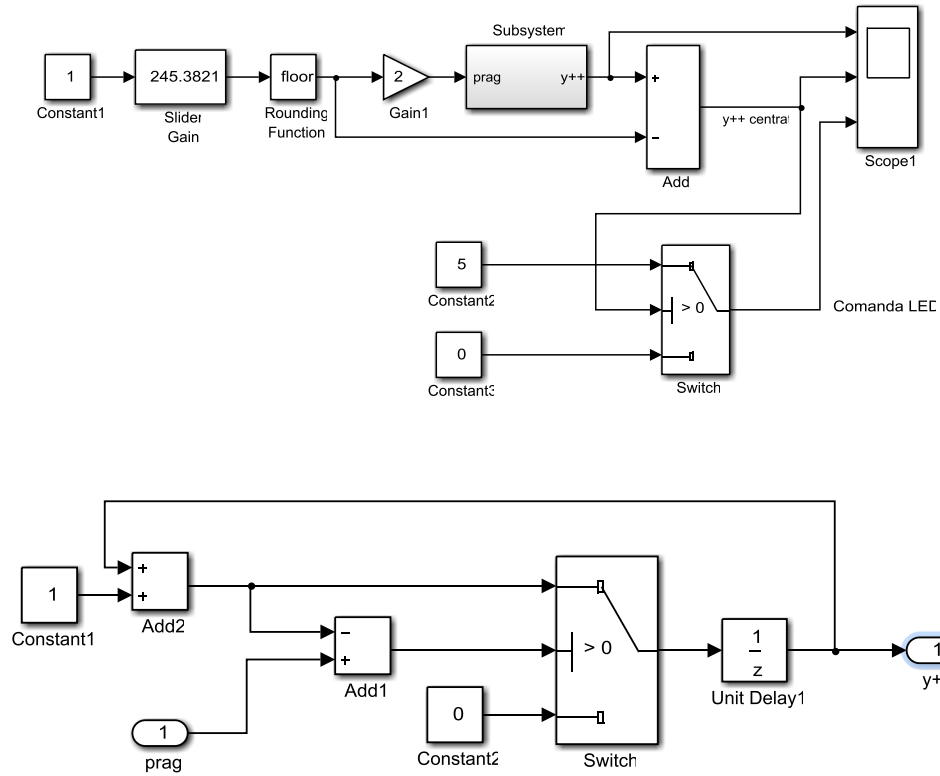


Fig. A5.16.c Schema de principiu Simulink echivalentă pentru implementarea aplicației din fig. A5.16b (sus); Detaliu de proiectare a subsistemului (jos)

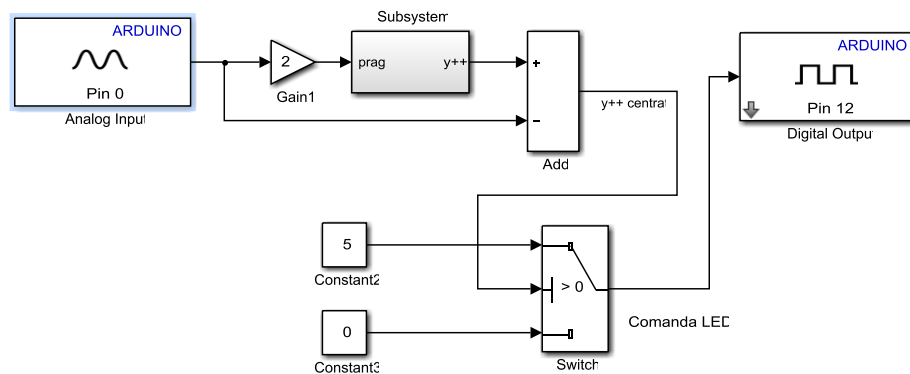


Fig. A5.16d Schema Simulink finală pentru implementarea în timp real

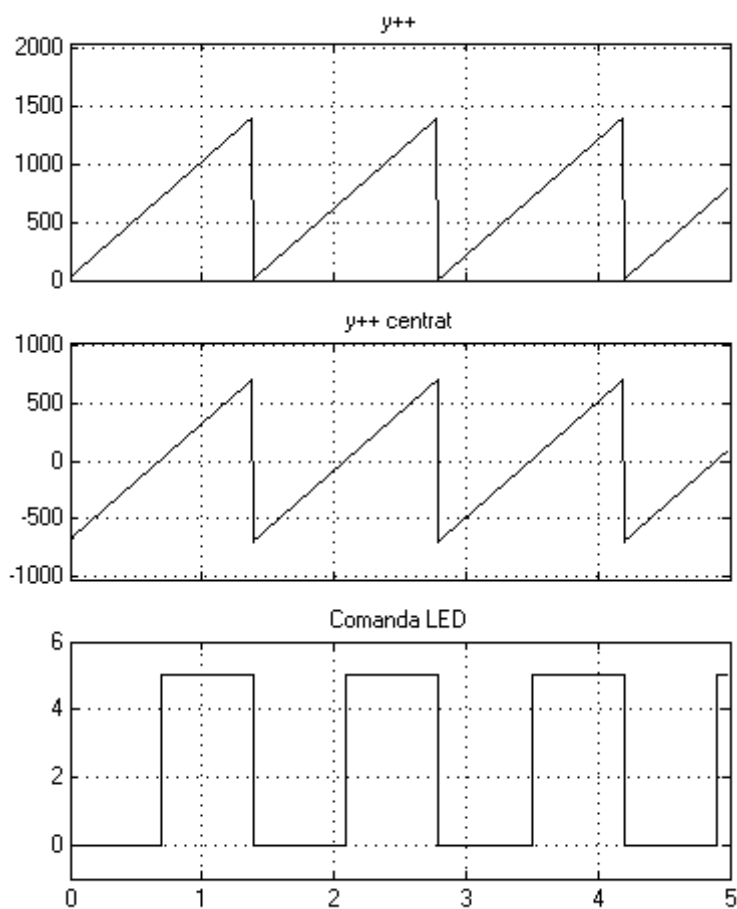
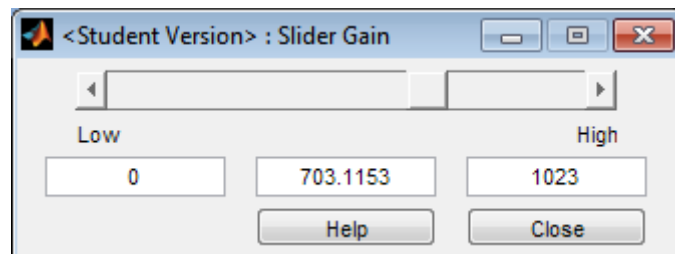


Fig. A5.16e Rezultate de simulare pentru modelul din fig. A5.16c: poziția potențiometrului (sus);
forme de undă (jos)

Evident, soluția de implementare nu este unică. Utilizatorul poate combina în mod creativ diferite module din biblioteca Simulink, în scopul obținerii unei variante optime.

Monitorul serial

Comunicarea dintre placa Arduino și computer este în esență serială, prin intermediul interfeței *USB* (fig. A5.17). Aceasta furnizează o modalitate flexibilă de transmitere (*TX*), respectiv recepționare (*RX*) a informației de la/ către platforma Arduino, permițând descărcarea programelor în vederea executării lor de către microcontroller, recepționarea unor mesaje de stare de către computer, depanarea programelor, etc.

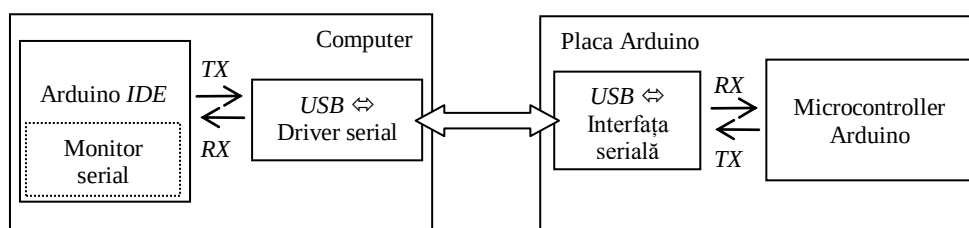


Fig. A5.17 Comunicarea serială dintre PC și platforma Arduino

Datele sunt trimise, respectiv recepționate pe rând, pe câte un singur fir, sub formă de impulsuri electrice reprezentând secvențe de biți. Acestea sunt de obicei structurate în seturi de câte opt biți, numite *bytes* (fig. A5.18).

Unele versiuni de plăci Arduino (ex. Arduino Mini, Boarduino, ș.a.), conțin separat modulul interfață *USB* (numit adaptor *USB*), și un alt modul distinct pe care se află microcontrollerul. Principiul de operare rămâne în esență același.

Monitorul serial (*Serial Monitor*), este o componentă a *Arduino IDE* care face legătura dintre computer și placa Arduino, permițând utilizatorului să transmită sau să vizualizeze pe ecranul computerului diferite mesaje, rezultate de implementare, și în general, să controleze programul de la tastatură.

Pentru accesarea monitorului serial, se face dublu click pe icoana dedicată, aflată în partea dreaptă a ferestrei de editare *Arduino IDE* (v.fig. A5.6). Se deschide un ecran ca cel din fig. A5.19, unde pot fi afișate diferite mesaje utilizate de programul curent. De asemenea, prin intermediul acestuia se pot transmite date către Arduino, prin introducerea de text în spațiul dreptunghiular (*text-box*) aflat în dreptul butonului *Send*.

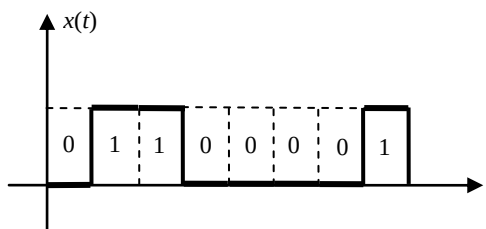


Fig. A5.18 Exemplu de succesiune de biți (0-logic / 1-logic) folosită la transmisia serială, reprezentând caracterul ASCII 'a'.

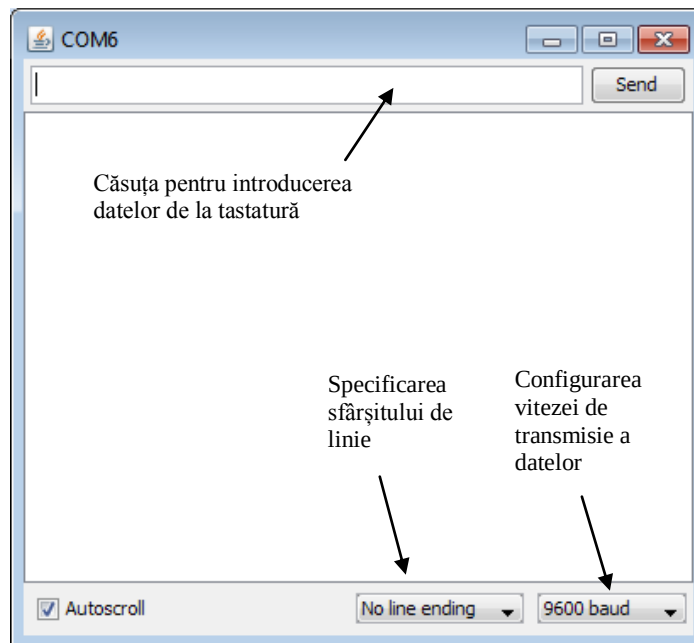


Fig. A5.19 Ecranul specific monitorului serial

Viteza de transmisie a datelor (*baud rate*), măsurată în biți pe secundă (*baud*) poate fi selectată din căsuța (*drop-down box*), localizată în partea din dreaptă - jos a monitorului serial. De asemenea, există posibilitatea de configurare a sfârșitului de linie prin transmiterea automată a comenzilor rând nou (*Carriage return*), linie nouă (*Newline*), ambele (*Both NL and CR*) sau nici una (*No line ending*), în căsuța corespunzătoare (v. fig. A5.19).

Pentru comunicarea prin portul serial, se utilizează biblioteca Arduino serială (*Arduino Serial library*). Ea cuprinde funcții specializate care facilitează transmiterea și recepționarea datelor. În general, informația vehiculată constă în seturi de variabile structurate grupat, în *mesaje*. Acestea conțin caractere speciale care identifică începutul (*header*) sau sfârșitul (*footer*) setului de date. Mesajele pot fi de tip text sau în format binar.

În fig. A5.20 se prezintă un exemplu de program prin care se dă comandă sistemului Arduino să afișeze un text oarecare (Ex. "Arduino, ") pe ecranul monitorului serial. Astfel, în mod tipic în cadrul funcției `setup()`, se utilizează instrucțiunea `Serial.begin(9600)` pentru configurarea vitezei de transmisie (9600 baud este valoarea cea mai des folosită în aplicații, însemnând aproximativ 1000 de caractere pe secundă). Pentru afișarea textului, se apelează instrucțiunea `Serial.print("<mesaj>")`, unde `<mesaj>` se introduce între ghilimele duble (") - excepție făcând numerele, care nu trebuie incluse între ghilimele. Dacă se dorește afișarea pe linii separate se apelează instrucțiunea `Serial.println()`, având aceeași

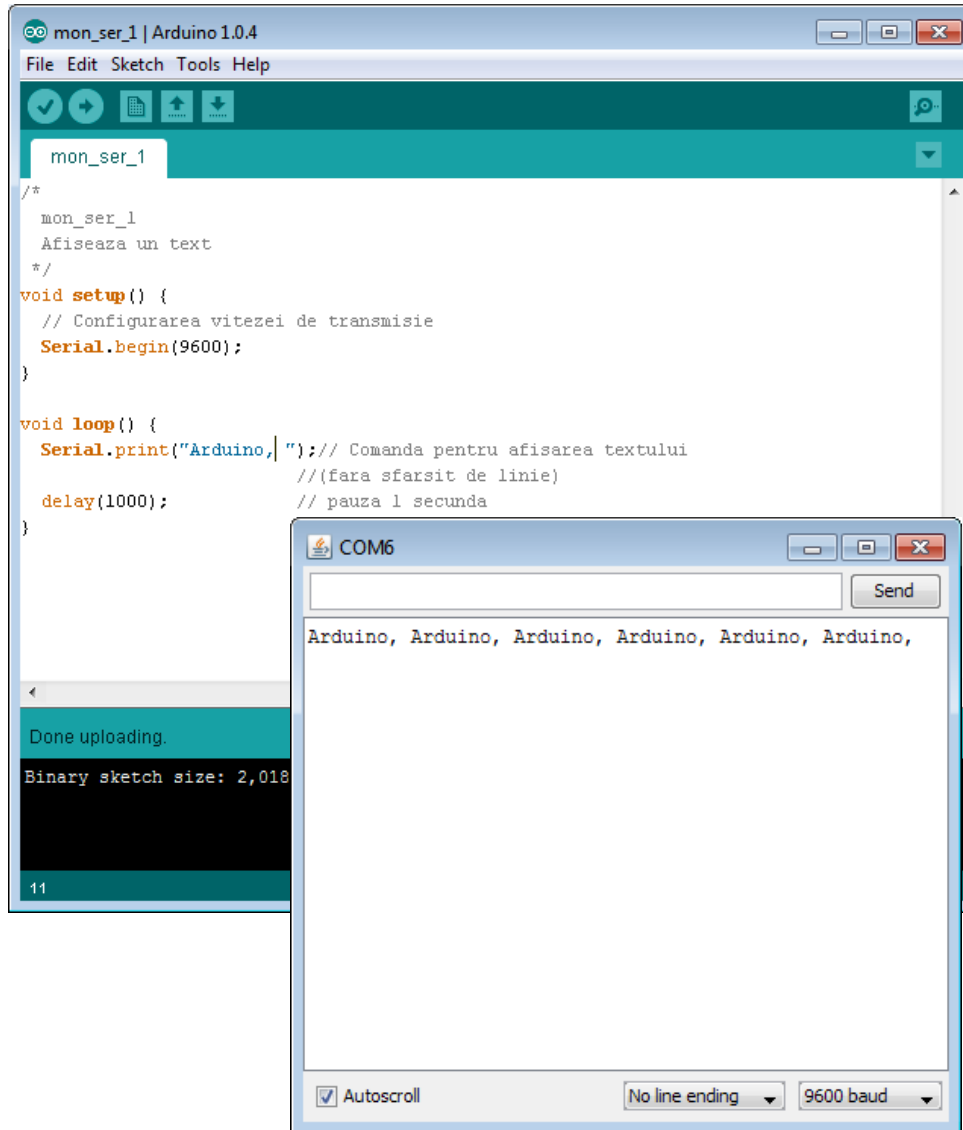


Fig A 5.20 Afişarea unui text la monitorul serial

sintaxă. Datele seriale trimise de sistemul Arduino, vor apare pe ecran, în formatul specificat. *LED*-ul *TX* de pe placă, va semnaliza transmiterea acestora la intervale egale, prescrise de instrucţiunea *delay* (aici 1000 ms). În cazul în care viteza de transmisie configurată în program diferă de cea selectată în căsuţa monitorului serial, textul afişat diferă de cel real. De aceea trebuie verificat de fiecare dată ca cele două viteze, de transmisie, respectiv recepţie, să fie întotdeauna identice.

De remarcat faptul că datele pot fi afișate în diferite moduri, în funcție de tipul lor, folosind instrucțiunile `Serial.print()` și `Serial.println()`. În fig. A5.21 este prezentat un *sketch* simplu care demonstrează acest lucru. Se observă că variabilele declarate cu prefixele `int` și `byte` sunt afișate în mod identic, deoarece ambele desemnează un număr întreg, mai mic decât 2^8-1 (v. Anexa 1, Tabel A1.23). Pe de altă parte variabila cu prefixul `char` este afișată sub forma caracterului 'A' deoarece numărul 65 reprezintă codul *ASCII* (*American Standard Code for Information Interchange*) asociat acestuia (v. Anexa 1 Tabelul A1.31) În fine, prefixul `float` desemnează un număr cu zecimale, fiind afișat ca atare.

```

/*
  mon_ser_1a
  Afiseaza date de diferite tipuri
*/
char v_ch = 65;    // Declararea variabilelor de tip caracter
//ASCII(char)
int v_int = 65;    // intreg
byte v_byte = 65; // byte (octet- 8 biti)
float v_float = 65;// cu zecimale

void setup() {
  Serial.begin(9600);
}

void loop() { // Afisarea datelor la monitorul serial
  Serial.println(v_ch);
  Serial.println(v_int);
  Serial.println(v_byte);
  Serial.println(v_float);

  delay(2000); // Pauza de doua secunde
  // Incrementarea variabilelor
  v_ch++;
  v_int++;
  v_byte++;
  v_float+=1;
}

```

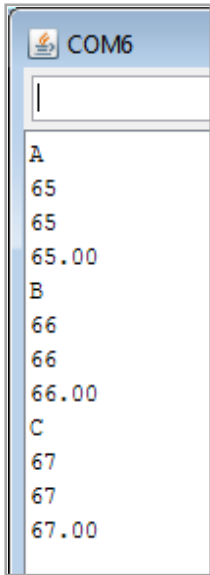


Fig. A5.21 Afișarea datelor în funcție de tipul declarat

De asemenea, instrucțiunile `Serial.print()` și `Serial.println()` se utilizează și pentru trimiterea datelor de la senzorii conectați la placa Arduino la computer, ca în exemplul din fig. A5.22a. Pe ecranul monitorului serial for fi afișate în timp real valorile eșantioanelor achiziționate. Trebuie precizat că există diferite variante de afișare a datelor recepționate de la sistemul Arduino, în funcție de tipul specificat în argumentul acestor funcții (Tabelul A5.1, fig. A5.22b).


```

void setup()
{
    Serial.begin(9600); // setează rata transmisiei
}
void loop()
{
    Serial.println(analogRead (0)); // citește o valoare
    delay(2000); // analogică de la pinul 0 și o trimite la
    // monitorul serial apoi așteaptă 2 secunde
}

```

COM4

1023
1023
950
951
951
805
696
646
520
414
204

Fig. A5.22a Citirea datelor de la senzor și afișarea la monitorul serial, pentru exemplul din fig. A5.16a (potențiometrul la intrarea analogică A0)

Tabelul A5.1 Formate de afișare a unei variabile întregi *val* folosind instrucțiunile `Serial.print()`, `Serial.println()`. (Ex. *val*=1023)

Nr. crt.	Sintaxă	Format de afișare
1.	<code>Serial.print(val)</code>	Numărul întreg recepționat Ex. 1023
2.	<code>Serial.print(val, DEC)</code>	Același format ca mai sus. Numărul format din cifre în baza 10
3.	<code>Serial.print(val, HEX)</code>	Format hexazecimal (baza 16) Ex. 3FF
4.	<code>Serial.print(val, BIN)</code>	Format binar (baza 2) Ex. 111111111

```

* Citirea datelor si
trimiterea la computer, sub diferite formate*/
void setup()
{
    Serial.begin(9600); // setează rata transmisiei
}
void loop()
{
    Serial.println(analogRead (0)); // un număr între 0 și 1024
    Serial.println(analogRead (0), DEC); // la fel ca mai sus
    Serial.println(analogRead (0), HEX); // reprezentarea
    // hexazecimala
    Serial.println(analogRead (0), BIN); // reprezentare binară
    // (aici 10 biți)
    delay(2000);
}

```

COM4

1023
3FF
111111111
1023
1023
3FF
111111111
1023
1023
3FF
111111111

Fig. A5.22b Afișarea sub diferite formate a poziției maxime a potențiometrului pentru exemplul din fig. A5.16 a

Următorul exemplu (fig. A5.23) ilustrează modalitatea prin care se pot trimite date de la computer către Arduino, folosind comunicarea serială. În funcție de datele primite

```
/*
mon_ser_2
Citirea seriala a datelor de catre Arduino
*/
int d;                // declararea variabilei globale d - intarziere
void setup() {
  Serial.begin(9600);    // Configurarea vitezei de transmisie
  pinMode(13,'OUTPUT');  // Configurarea pinului 13 ca iesire digitala
}

void loop() {
  // Verifica daca cel puțin un caracter este disponibil
  if (Serial.available()){
    // Citeste de la tastatura si memoreaza in variabila c
    char c = Serial.read();
    d=c*10;                // Calculul intarzierii in functie de tasta apasata
    Serial.println(d);      // Afisarea valorii lui d
  }
  digitalWrite(13, HIGH);delay(d);  //LED-ul va clipi la intervale
  digitalWrite(13, LOW);delay(d);    // prescrise de variabila d
}
```

Fig. A5.23 Exemplu pentru recepția serială a datelor de către sistemul Arduino

(aici valoarea *ASCII* a tastei introduse de la consolă), sistemul își va ajusta în timp real variabilele pentru comanda elementului de execuție (ex. aprinderea/stingerea *LED*-ului conectat la un pin Arduino, cu o viteză dată de caracterul introdus). Se observă utilizarea funcției `Serial.available()` care testează dacă a fost introdus un caracter, fără a întrerupe programul. De asemenea, comanda `Serial.read()` este de obicei folosită pentru citirea serială a datelor de intrare de către sistemul Arduino.

Revenind la sketch-ul din fig. A5.22a, se poate remarca faptul că datele primite de la senzor (aici tensiunea prescrisă de potențiometru) pot fi mai departe memorate și afișate în timp real cu ajutorul unui program Matlab simplu, ca cel prezentat în fig. A5.24.

Astfel, instrucțiunea `serial` construiește un *obiect* `s` asociat cu portul `'COM4'`, iar pentru a conecta acest obiect la portul serial se apelează instrucțiunea `fopen(s)`. Comanda `fscanf(s,%d)` citește datele achiziționate și le memorează într-o matrice `'potentiometru'` de numere întregi (`%d`), cuprinse în intervalul 0-1023. Se face apoi o scalare prin înmulțire cu $5V/1023$ pentru a transforma valorile obținute în volți. Instrucțiunea `stem` este similară cu `plot`, cu deosebirea că datele vor fi proiectate pe axa orizontală (aspect de eșantioane). De asemenea, comanda `drawnow` actualizează fereastra figurii la fiecare pas, permițând vizualizarea în timp real. În fine, ultima instrucțiune `fclose(s)` comandă închiderea portului `s`.

```

% Afiseaza in timp real tensiunea prescrisa de potentiometru
clear all;close all;clc;
delete(instrfindall);
s=serial('COM4');
fopen(s);
N=20; % Nr. de esantioane
potentiometru=zeros(1,N); % memoreaza datele citite
figure;
    hold on;
    title('Date citite de la senzor');
    xlabel('Nr. esantion');...
    ylabel('Tensiunea [V]');...
    axis([0,N,-0.1,5.1]);
    for i=1:N,
        potentiometru(i)=fscanf(s,'%d');
        potentiometru(i)=potentiometru(i)*5/1023;%Transforma in
volti
        stem(i,potentiometru(i),'-*');grid on;
        drawnow; % actualizeaza fereastra figurii
    end
    pause(2);
    fclose(s);

```

Fig. A5.24 Program Matlab pentru memorarea și afișarea în timp real a datelor citite la o intrare analogică Arduino

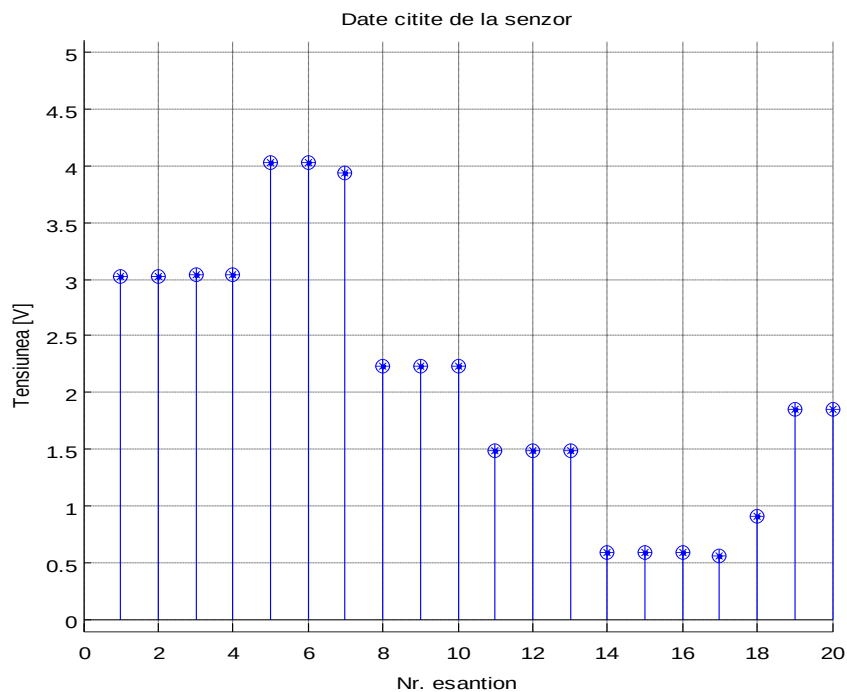


Fig. A5.25 Exemplu de afișare a datelor pentru programul din fig. A.5.24